

Rota Vital — Tradução Comentada C ↔ Java

Algoritmos e Estruturas de Dados — Unidade 1

Histórico de Operações (Pilha)

Responsável: Andrews Queiroz

A pilha do histórico foi implementada em C nos arquivos `pilha.c/pilha.h` como uma lista encadeada, onde a struct `NoOperacao` guarda uma `Operacao` e um ponteiro proximo apontando para o nó inserido antes dela; em Java, essa mesma relação vira a classe interna privada `NoOperacao`, dentro de `PilhaHistorico`, com uma referência proximo no lugar do ponteiro. A struct `Operacao` (campos `sequencia`, `tipo`, `referencia`, `quantidade`, `data` e `descricao`) foi traduzida diretamente para a classe `Operacao.java`, um POJO com os mesmos campos, agora encapsulados por getters e setters. O enum `TipoOperacao` de `pilha.h` (`ENTRADA_ESTOQUE`, `REMOCAO_ESTOQUE`, `RETIRADA_ESTOQUE`, `REQUISICAO_RECEBIDA`, `REQUISICAO_ATENDIDA`, `REQUISICAO_RECUSADA`) foi reimplementado como um enum Java de mesmo nome, cada valor carregando o texto de exibição que em C ficava na função `nome_operacao()`. As operações `pilha_empilhar` e `pilha_desempilhar`, que em C recebem o ponteiro da pilha como primeiro parâmetro e retornam um código do enum `ResultadoPilha` (`PILHA_OK`, `PILHA_VAZIA`, etc.), viraram os métodos de instância `empilhar(Operacao)` e `desempilhar()` da classe `PilhaHistorico`; como Java não tem código de retorno de erro como convenção, o caso `PILHA_VAZIA` foi traduzido para o lançamento da exceção customizada `PilhaVaziaException`, e entradas inválidas — antes barradas na validação interna de `pilha_empilhar` — agora lançam `IllegalArgumentException`. Em ambas as versões, a lógica central de push/pop no topo (o novo nó aponta para o antigo topo, que passa a ser o segundo elemento) foi mantida idêntica, o que os testes `teste_pilha.c` e `TestePilha.java` confirmam ao repetirem os mesmos casos (criar, empilhar, desempilhar em ordem LIFO, consultar topo e contar por tipo) com o mesmo resultado nas duas linguagens.

Tradução Estoque

Responsável: Hilton Resende Montes Neto

A transição do paradigma estruturado em C para o orientado a objetos em Java no módulo de estoque do sistema RotaVital eliminou o gerenciamento manual de baixo nível por meio de mapeamentos diretos no código. A estrutura passiva struct NoEstoque do arquivo estoque.c, que apenas agrupava registros, evoluiu para a classe interna estática NoEstoque dentro de Estoque.java, encapsulando o estado e permitindo que o objeto ItemEstoque fosse protegido por métodos de validação como validar(item). O gerenciamento manual de memória executado em C via malloc(sizeof(NoEstoque)) na função estoque_inserir, que exigia a liberação manual por meio de loops com free(atual) na função estoque_destruir para evitar memory leaks, foi integralmente substituído em Java pelo operador new NoEstoque(...) dentro do método inserir(ItemEstoque item), delegando a desalocação ao Garbage Collector. Por fim, o ponteiro aritmético explícito NoEstoque *proximo, utilizado nos laços while(atual != NULL) em C para navegar fisicamente pelos endereços de memória da lista, transformou-se na referência de objeto NoEstoque proximo em Java, permitindo que a varredura nos métodos listar() e totalDisponivel(...) ocorra de forma fortemente tipada e segura contra falhas de segmentação (segmentation faults).

Tradução Fila

Responsável: Glauco Santos Seixas Cerqueira

Em `fila.h`, `FilaRequisicoes` tem `inicio` e `fim` como ponteiros para `NoRequisicao`; em `FilaRequisicoes.java`, são referências à classe interna `NoRequisicao`. Em `fila.c`, `fila_enfileirar(...)` virou `enfileirar(...)`: cria o nó com `new`, liga em `fim.proximo` (ou `inicio`, se vazia), atualiza `fim` e retorna o ID. `fila_desenfileirar(...)` virou `desenfileirar()`: avança `inicio`, zera `fim` ao esvaziar e retorna a `Requisicao`. Em C, `free` libera o nó; em Java, a coleta de lixo o faz. FIFO e $O(1)$ em ambas. Quanto aos erros, o enum `ResultadoFila` do C (`FILA_PARAMETRO_INVALIDO`, `FILA_VAZIA`) foi substituído por exceções: parâmetros ou dados inválidos lançam `IllegalArgumentException`, e tentar `desenfileirar` uma fila vazia lança `FilaVaziaException`, o equivalente Java a checar `fila->inicio == NULL` antes de prosseguir.